

Scoring & sentiment methodology

Formulas, weights, and UI consistency

April 2026 · Companion to the technical white paper

CANONICAL FORMULA REFERENCE

This document is the canonical reference for how **AI Visibility**, **Share of Voice**, **position**, **sentiment**, and the **composite AI Presence Index** are computed, and how **sentiment breakdown** relates to those numbers.

1. AI Visibility Score

- **Definition:** Share of engine runs (one run = one prompt × one model/engine) where the target brand is mentioned.

- **Formula:**

$$(\text{number of runs with brandMentioned}) / (\text{total runs}) \times 100$$

2. Share of Voice (mention share)

- **Definition:** Target brand's mention volume vs all tracked entities' mentions in the same run set.

- **Formula:**

$$(\text{sum of mentions on target brand entities}) / (\text{sum of mentions on all entities}) \times 100$$

"Mention" definition: One mention = one occurrence of a brand name within a single entity row. If a brand's entity row carries `mentions: 3` (or `mentionCount: 3`) for a given run (appearing three times in that answer), it contributes 3 to both the numerator and denominator. Binary presence (mentioned/not) is NOT used here — raw occurrence count is.

- **Table:** Each row shows **mentions**, **SOV %**, **average position** (where available), and a **sentiment index 0–100** (see §6).
-

3. Position Score

- For each run where the brand is mentioned and `brandEntity.positionRank` exists, contribute the reciprocal rank: $1 / \text{rank}$.

- **Formula (raw):** Average of $1/\text{rank}$ values across qualifying runs.

- **Normalization to 0–100:**

$$\text{positionScore} = (\text{avgReciprocalRank} / \text{MAX_RECIPROCAL_RANK}) \times 100$$

where `MAX_RECIPROCAL_RANK = 1.0` (rank 1 = best possible = score 100).

This means: rank 1 → 100, rank 2 → 50, rank 5 → 20, rank 10 → 10.

- **Runs without `positionRank`:** Excluded from both numerator and denominator. They do NOT count as worst rank. If zero qualifying runs exist, `positionScore = 0`.
 - **Thin data note:** If fewer than 3 runs have a `positionRank`, treat `positionScore` with low confidence. The composite still uses it at full 20% weight — see §5.
-

4. Sentiment score (composite / AI Presence Index)

Sentiment is **not** “percentage positive” in this path. It uses **numeric weights** per label, then maps the average to 0–100.

4.1 Label weights (internal, on [-1, +1] scale)

Defined as `SENTIMENT_VALUES` in `scoringEngine.js`:

Label	Weight
positive	+1
neutral	+0.25 (slightly above zero so “no opinion” is not a full penalty)
negative	−1
missing / unknown	treated as 0 (via <code>getSentimentWeight</code>)

Missing vs neutral distinction:

- In the **composite** (§4.3): `missing/unknown` sentiment → weight **0** (does not pull the score toward positive or negative).
- In **breakdown counts** (§6, `computeSentimentBreakdown`): `missing/unknown` is bucketed as **“neutral”** for display purposes only. This is intentional — the breakdown is a human-readable summary; the composite needs a numerically neutral zero contribution from unknowns, not an artificial 62.5 boost.

4.2 Which runs count?

Only runs where:

- `run.brandMentioned === true`, and
- `run.brandEntity` exists.

Each such run **contributes exactly one sample** for the composite: `getSentimentWeight(brandEntity.sentiment)`. Breakdown bucketing uses `getSentimentBucket` on the same field.

4.3 Formulas

Let (w_i) be the weight for run (i), and (n) the number of qualifying runs.

1. Raw average (on [-1, 1]):

$$\text{sentimentRaw} = \frac{1}{n} \sum_i w_i$$

2. 0–100 score (composite component):

$$\text{sentimentScore} = \frac{\text{sentimentRaw} + 1}{2} \times 100$$

So “all positive” → raw 1 → **100**; “all negative” → raw −1 → **0**; “all neutral” → raw 0.25 → **62.5** (rounded in output).

5. AI Presence Index (overall score)

Weighted blend of the four components:

Component	Weight	Rationale
Visibility	30%	Primary reach signal — is the brand showing up at all?
Share of Voice	30%	Competitive context — how dominant vs peers?
Position	20%	Quality of mention — top-of-answer vs buried
Sentiment	20%	Perception — does the mention help or hurt?

Formula:

`overall = round(Vis×0.30 + SOV×0.30 + Pos×0.20 + Sent×0.20)` (clamped 0–100)

Thin-data handling: All four components always carry their full weight regardless of how many runs qualified for each. If Position has only 1 qualifying run, it still contributes 20%. This is a known trade-off favouring simplicity over statistical robustness. A future version may scale component weight by `min(qualifyingRuns/10, 1)`.

Weight rationale: 30/30/20/20 was set by product consensus (April 2026). Visibility and SOV are weighted equally as the two primary reach metrics; Position and Sentiment are secondary quality signals. Review annually.

6. Sentiment breakdown vs SOV sentiment vs composite sentiment

Comparability (target vs competitors): Target brand and competitor sentiment indices in the SOV table now both use **one vote per engine run** in which that brand appears (dominant entity row for that name per run), with the same `getSentimentWeight` → `((\text{avg}+1)/2×100)` mapping. They remain **not identical** to the composite breakdown when the **source field** differs: target uses `brandEntity.sentiment` when present; competitors use the **dominant entities[] row** for that key in each run (no `brandEntity` for competitors).

Surface	What it uses
Sentiment breakdown (<code>computeSentimentBreakdown</code>)	Same qualifying runs as composite for the target: <code>brandMentioned</code> + <code>brandEntity</code> ; buckets via <code>getSentimentBucket</code> ; <code>summary</code> includes percentages and rawCounts for exact KPI math.
SOV table — your brand	<code>getSentimentWeight(brandEntity.sentiment)</code> averaged over runs with <code>brandMentioned</code> + <code>brandEntity</code> (fallback: mention-weighted target entities if none).
SOV table — competitors	Per run where the competitor appears: <code>getSentimentWeight</code> on the highest-mention entity row for that name; then same <code>((\text{avg}+1)/2×100)</code> mapping.
Composite sentiment (§4)	Identical basis to target SOV when using <code>brandEntity</code> : one vote per qualifying run.

Fallback (target brand only): If there are no runs with `brandMentioned` + `brandEntity`, the target row falls back to mention-weighted aggregation from **target-named** `entities` only (same weight table).

7. Sentiment & Geo page — “Avg Sentiment” KPI

The **Avg Sentiment** KPI is a **dashboard-only** blend aligned with backend neutral mapping (62.5).

In `SentimentGeoPage.jsx`:

- Weights: **positive** = 100, **neutral** = 62.5, **negative** = 12 (aligned with backend: neutral raw 0.25 → $((0.25+1)/2 \times 100 = 62.5)$).

- Formula:**

```
avgSentiment = round((p×100 + n×62.5 + neg×12) / (p + n + neg))
```

where `p`, `n`, `neg` are **raw counts** from `sentiment.summary.rawCounts` when the API provides them.

Using raw counts instead of rounded percentages avoids compounding rounding errors. If only percentage values are available from the API, the UI falls back to `summary.positive` / `neutral` / `negative`; note ±1% error is possible due to upstream rounding.

- Backend composite / SOV / breakdown** follow §4–§6 (`SENTIMENT_VALUES` + per-run sampling for the target).
- This KPI is a weighted index** for quick scanning on that page.

If you need one number to match the scoring engine exactly, use **composite** `components.sentiment` or the SOV target **sentiment** from the API, not this KPI alone.

8. Query-level sentiment score (prompt tracking)

`computeQueryTracking` uses a **per-prompt display metric** across engines that returned a real sentiment label:

Signal	Formula
Positive rate	<code>engines with positive / engines with label</code>
Negative rate	<code>engines with negative / engines with label</code>
Neutral rate	<code>engines with neutral / engines with label</code>
Score (0–100)	<code>round(positiveRate×100 + neutralRate×62.5 + negativeRate×12)</code>

The API exposes `sentimentPositivePct`, `sentimentNeutralPct`, `sentimentNegativePct`, and `sentimentScore` per query row. The scalar score matches the §7 display weights for visual consistency.

Why not the §4 weight table here? Query tracking is a fast scan surface — the three explicit rates are more actionable than a single composite. The scalar score is provided for sorting/ranking rows.

9. Quick consistency checklist

1. **Same run set:** Breakdown `total` should equal the number of runs used in composite sentiment denominator when every such run is counted (target `brandEntity` path).
 2. **Target brand:** SOV sentiment index should move in lockstep with breakdown bucket mix when labels are stable—same runs; composite uses weights, breakdown uses buckets.
 3. **Competitors:** SOV competitor sentiment uses per-run dominant-entity labels; a gap vs your brand can still reflect **different fields** (`entities[]` vs `brandEntity`) or mention dominance within a run, not necessarily a bug.
 4. **Sentiment & Geo Avg %:** Intentionally a display blend (\$7), not identical to \$4.3.
-

10. Code map

Concern	Location
Visibility, SOV, position, composite sentiment, AI Presence Index	<code>computeVisibilityScore</code>
SOV table + target/competitor sentiment	<code>computeShareOfVoice</code>
Breakdown counts + summary % + <code>rawCounts</code>	<code>computeSentimentBreakdown</code>
Per-engine / per-category visibility	<code>computePerEngine</code> , <code>computePerCategory</code>
Per-prompt positive/neutral/negative rates + scalar score	<code>computeQueryTracking</code>
Sentiment & Geo KPI blend	<code>deriveSentimentAndGeo</code> in <code>SentimentGeoPage.jsx</code>

End of document.